

Do procedural à orientação a objetos

COBOL como ponto de partida · aplicação
bancária





Objetivo do encontro

Preservar a experiência em processos

Acrescentar elementos e responsabilidades

Distinguir classe, objeto, atributo e tipo

Organizar o projeto com pacotes e imports



COBOL como ponto de partida

DATA DIVISION descreve dados

PROCEDURE DIVISION organiza o fluxo

Nível 01, PIC e nível 88 são referências conhecidas

Processos continuam existindo em Java



A pergunta muda

Procedural: quais passos executar?

OO: quais elementos participam?

Verbos ajudam a localizar processos

Substantivos ajudam a modelar o domínio



Java não é COBOL com outra sintaxe

Tradução literal

Modelagem em Java

Troca palavras-chave

Representa o domínio

Mantém tudo em uma classe

Distribui responsabilidades

Concentra responsabilidades

Mantém o fluxo dentro dos métodos



Classe e grupo de dados

COBOL: nível 01

Java: classe

Descreve uma estrutura

Define um conceito e um tipo

Agrupar campos PIC

Agrupar estado e comportamento

Área recebe valores concretos

Origina vários objetos



Atributos descrevem o estado

numero e agencia pertencem a Conta

ativa representa a situação

saldo representa valor financeiro

Cada objeto possui seus próprios valores



Tipos comunicam significado

Texto, quantidade, condição, dinheiro e data

Identificador não é quantidade

O tipo restringe valores possíveis

O tipo não substitui a regra de negócio



PIC X(n) e String

PIC X(n): quantidade definida de posições

String: texto sem limite automático na declaração

Número de conta pode ser String

String é classe e tipo por referência



PIC 9(n), int e long

COBOL

PIC 9(3): três dígitos

PIC S9: aceita sinal

Capacidade declarada em dígitos

Java

int: inteiro de 32 bits

long: inteiro de 64 bits

Inteiros já são sinalizados



Nível 88 e boolean

COBOL

PIC X(1)

88 ATIVA VALUE 'S'

88 INATIVA VALUE 'N'

Outros caracteres ainda cabem no campo

Java

boolean ativa

Somente true ou false

Tipo primitivo

Mesma intenção, mecanismo diferente



Dinheiro e datas pedem tipos próprios

PIC S9(n)V99 COMP-3: decimal COBOL

BigDecimal: precisão decimal em Java

PIC 9(8): data representada por dígitos

LocalDate: tipo com significado de calendário



Primitivos e tipos por referência

Primitivos: byte, short, int, long

float, double, char e boolean

Referência: String, BigDecimal e LocalDate

Classes do domínio também são tipos



Classe, objeto e referência

Conta: classe e tipo

`new Conta()`: cria um objeto

`conta`: variável que guarda a referência

Objetos com o mesmo estado ainda podem ser distintos



Responsabilidades no domínio bancário

Cliente: CPF e nome

Conta: número, agência, situação e saldo

Transacao: contas, valor e data

Banco coordena; AplicacaoBanco inicia



O fluxo continua existindo

Métodos ainda executam instruções em sequência

AplicacaoBanco inicia o sistema

Banco coordena a operação

Transacao relaciona objetos Conta



Pacotes organizam e identificam

br.com.curso.banco

br.com.curso.banco.dominio

A estrutura de pastas acompanha o pacote

Nome completo: br.com.curso.banco.dominio.Conta



import e COPY

import

Resolve o nome simples da classe

Não copia nem baixa código

Mesmo pacote dispensa import

COPY

Insere conteúdo do copybook

Mecanismo diferente

Semelhança: organização e reutilização



Demonstração no VS Code

Criar o pacote dominio

Criar Cliente, Conta e Transacao

Escolher tipos e adicionar imports

Criar objetos em AplicacaoBanco

Compilar e executar



Fechamento

A experiência procedural continua útil

Classes e objetos representam o domínio

Tipos registram significado

Responsabilidades evitam concentração

Pacotes e imports sustentam a organização